Laravel Product CRUD

SOFT DELETE / TRASH / RESTORE / PERMANENT DELETE / IMAGE WATERMARK

A practical classroom guide using Laravel, Blade, a database (MySQL or SQLite), and Intervention Image v4. The full ProductController appears in the following pages.

What you will build

- Add, view, edit, and paginate products (10 items per page).
- Upload an image, resize it, add a logo watermark, and convert it to WebP.
- Soft-delete a product into Trash without deleting its image.
- Restore products, or permanently delete both the record and its image.

Files covered in this PDF

- Installation commands and required watermark file
- app/Http/Controllers/ProductController.php - full controller
- app/Models/Product.php - SoftDeletes trait
- database/migrations/...create_products_table.php - deleted_at column
- routes/web.php - CRUD, Trash, Restore, and Permanent Delete

Quick concept

Soft delete writes a timestamp to deleted_at. Active queries exclude that record. onlyTrashed() lists deleted records, restore() makes them active again, and forceDelete() deletes the row permanently.

1 | Installation and setup

System requirements

Use an existing Laravel application, PHP 8.3+, and the PHP GD extension with WebP support (or another supported Intervention image driver). Composer must be installed.

Check your PHP version and extensions

```
php -v
php -m
```

Install Intervention Image v4 Laravel integration

Run in the project directory

```
composer require intervention/image-laravel:^4.0
php artisan vendor:publish --provider="Intervention\Image\Laravel\ServiceProvider"
composer dump-autoload
php artisan optimize:clear
```

The intervention/image-laravel package provides the Image facade automatically. The default image driver is GD. The provider publish command is optional but useful if you need to configure the image driver.

Generate the model, migration, and controller (only if missing)

Generate starter files

```
php artisan make:model Product -m
php artisan make:controller ProductController --resource
```

Watermark image

Create a transparent PNG logo at public/images/watermark.png. The filename must match the controller exactly. The code overlays the logo in the lower-right corner at 55% opacity.

Database, public storage, and run

After setting database credentials in .env

```
php artisan migrate
php artisan storage:link
php artisan serve
```

Existing products table? Do not run another create_products_table migration. Instead create a new migration to add deleted_at with $table->softDeletes().

2 | Complete ProductController.php

Copy the code below into app/Http/Controllers/ProductController.php. The listing continues automatically across pages; all methods are included.

app/Http/Controllers/ProductController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Product;
use Illuminate\Http\Request;
use Illuminate\Http\UploadedFile;
use Illuminate\Support\Facades\Storage;
use Illuminate\Support\Str;
use Intervention\Image\Alignment;
use Intervention\Image\Format;
use Intervention\Image\Laravel\Facades\Image;

class ProductController extends Controller
{
    // 1. DISPLAY ALL ACTIVE PRODUCTS
    public function index()
    {
        $products = Product::latest()->paginate(10);

        return view('products.index', compact('products'));
    }

    // 2. SHOW CREATE FORM
    public function create()
    {
        return view('products.create');
    }

    // 3. CREATE PRODUCT AND APPLY WATERMARK
    public function store(Request $request)
    {
        $data = $request->validate([
            'name' => ['required', 'string', 'max:150'],
            'description' => ['nullable', 'string'],
            'price' => ['required', 'numeric', 'min:0'],
            'image' => [
                'required', 'image',
                'mimes:jpg,jpeg,png,webp', 'max:3072',
            ],
        ]);

        Product::create([
            'name' => $data['name'],
            'description' => $data['description'] ?? null,
            'price' => $data['price'],
            'image_path' => $this->saveWatermarkedImage(
                $request->file('image')
            ),
        ]);

        return redirect()->route('products.index')
            ->with('success', 'Product created. Watermark applied.');
    }

    // 4. DISPLAY ONE PRODUCT
```

app/Http/Controllers/ProductController.php / continued

```
    public function show(Product $product)
    {
        return view('products.show', compact('product'));
    }

    // 5. SHOW EDIT FORM
    public function edit(Product $product)
    {
        return view('products.edit', compact('product'));
    }

    // 6. UPDATE DETAILS AND OPTIONALLY REPLACE IMAGE
    public function update(Request $request, Product $product)
    {
        $data = $request->validate([
            'name' => ['required', 'string', 'max:150'],
            'description' => ['nullable', 'string'],
            'price' => ['required', 'numeric', 'min:0'],
            'image' => [
                'nullable', 'image',
                'mimes:jpg,jpeg,png,webp', 'max:3072',
            ],
        ]);

        $product->name = $data['name'];
        $product->description = $data['description'] ?? null;
        $product->price = $data['price'];

        $oldImage = null;
        $newImage = null;

        if ($request->hasFile('image')) {
            $newImage = $this->saveWatermarkedImage(
                $request->file('image')
            );
            $oldImage = $product->image_path;
            $product->image_path = $newImage;
        }

        try {
            $product->save();
        } catch (\Throwable $e) {
            // Clean up a new file if saving to DB fails.
            if ($newImage) {
                Storage::disk('public')->delete($newImage);
            }
            throw $e;
        }

        if ($oldImage) {
            Storage::disk('public')->delete($oldImage);
        }

        return redirect()->route('products.index')
            ->with('success', 'Product updated successfully.');
    }

    // 7. SOFT DELETE: SET deleted_at; KEEP IMAGE
    public function destroy(Product $product)
    {
        $product->delete();
```

app/Http/Controllers/ProductController.php / continued

```
118
119         return redirect()->route('products.index')
120             ->with('success', 'Product moved to Trash.');
121     }
122
123     // 8. DISPLAY ONLY SOFT-DELETED PRODUCTS
124     public function trash()
125     {
126         $products = Product::onlyTrashed()
127             ->latest('deleted_at')->paginate(10);
128
129         return view('products.trash', compact('products'));
130     }
131
132     // 9. RESTORE A PRODUCT FROM TRASH
133     public function restore(int $id)
134     {
135         $product = Product::onlyTrashed()->findOrFail($id);
136         $product->restore();
137
138         return redirect()->route('products.trash')
139             ->with('success', 'Product restored successfully.');
140     }
141
142     // 10. PERMANENTLY DELETE RECORD AND IMAGE
143     public function forceDelete(int $id)
144     {
145         $product = Product::onlyTrashed()->findOrFail($id);
146         $imagePath = $product->image_path;
147
148         $product->forceDelete();
149
150         if ($imagePath) {
151             Storage::disk('public')->delete($imagePath);
152         }
153
154         return redirect()->route('products.trash')
155             ->with('success', 'Product permanently deleted.');
156     }
157
158     // 11. WATERMARK HELPER: RETURNS STORED PATH STRING
159     private function saveWatermarkedImage(UploadedFile $file): string
160     {
161         $image = Image::decode($file);
162         $image->scaleDown(width: 1000);
163
164         $watermarkPath = public_path('images/watermark.png');
165         if (!is_file($watermarkPath)) {
166             throw new \RuntimeException(
167                 'Watermark not found: public/images/watermark.png'
168             );
169         }
170
171         $watermark = Image::decode($watermarkPath);
172         $watermarkWidth = min(
173             180,
174             max(1, (int) ($image->width() * 0.25))
175         );
176         $watermark->scaleDown(width: $watermarkWidth);
177
178         // Bottom-right, 12px padding, 55% opacity
```

app/Http/Controllers/ProductController.php / continued

```
        $image->insert(
            $watermark,
            12,
            12,
            Alignment::BOTTOM_RIGHT,
            0.55
        );

        $path = 'products/' . Str::uuid() . '.webp';
        $encoded = $image->encodeUsingFormat(
            Format::WEBP,
            quality: 80
        );

        if (!Storage::disk('public')->put(
            $path,
            (string) $encoded
        )) {
            throw new \RuntimeException('Failed to save image.');
        }

        return $path;
    }
}
```

3 | Product model

The SoftDeletes trait tells Eloquent to use deleted_at instead of physically deleting a product when delete() is called.

app/Models/Product.php

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

class Product extends Model
{
    use SoftDeletes;

    protected $fillable = [
        'name', 'description', 'price', 'image_path'
    ];
}
```

4 | Migration

This migration is for projects creating the products table for the first time.

database/migrations/...create_products_table.php

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    public function up(): void
    {
        Schema::create('products', function (Blueprint $table) {
            $table->id();
            $table->string('name', 150);
            $table->text('description')->nullable();
            $table->decimal('price', 10, 2);
            $table->string('image_path')->nullable();
            $table->timestamps();
            $table->softDeletes(); // adds deleted_at
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('products');
    }
};
```

5 | Routes

Put the Trash route before Route::resource so /products/trash is not mistaken for a product ID.

routes/web.php (merge with existing file)

```php
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\ProductController;

// Register this before the resource route.
Route::get('/products/trash',
    [ProductController::class, 'trash']
)->name('products.trash');

Route::patch('/products/{id}/restore',
    [ProductController::class, 'restore']
)->name('products.restore');

Route::delete('/products/{id}/force-delete',
    [ProductController::class, 'forceDelete']
)->name('products.force-delete');

Route::resource('products', ProductController::class);
```

6 | Test the application

Expected Blade views

The controller expects resources/views/products/index.blade.php, create.blade.php, show.blade.php, edit.blade.php, and trash.blade.php. Forms that upload images must include enctype="multipart/form-data".

Recommended demonstration flow

- Create a product with an uploaded JPG or PNG. Confirm that a watermarked WebP file appears under storage/app/public/products.
- Edit the product, change its price, and optionally upload a replacement photo.
- Delete the product. It vanishes from the active index, but deleted_at is populated.
- Open /products/trash. Confirm the deleted product is listed with its original image.
- Restore the product and confirm it returns to the active list.
- Soft-delete again, then permanently delete it. Confirm the record and image are removed.

Example Blade form requirements

Create form essentials

```
<form action="{{ route('products.store') }}"
      method="POST" enctype="multipart/form-data">
    @csrf
    <input name="name" required>
    <input name="price" type="number" step="0.01" required>
    <input name="image" type="file" accept="image/*" required>
    <button type="submit">Save Product</button>
</form>
```

Typical buttons for Trash

Inside a products/trash.blade.php loop

```
<form action="{{ route('products.restore', $product->id) }}" method="POST">
    @csrf
    @method('PATCH')
    <button type="submit">Restore</button>
</form>
<form action="{{ route('products.force-delete', $product->id) }}" method="POST">
    @csrf
    @method('DELETE')
    <button type="submit">Delete Forever</button>
</form>
```

7 | Common errors and notes

Class "Intervention\Image\Laravel\Facades\Image" not found

Verify that intervention/image-laravel is installed (not only intervention/image), then refresh Composer autoload and Laravel caches.

Troubleshooting commands

```
composer show intervention/image-laravel
composer dump-autoload
php artisan optimize:clear
```

Where does UploadedFile come from?

Illuminate\Http\UploadedFile is provided by Laravel and extends Symfony UploadedFile. No separate UploadedFile installation is needed.

Why does saveWatermarkedImage(...) return : string?

The method saves an image file but returns a stored relative path such as products/abc123.webp. The string path is saved in the database column image_path; the binary image lives in public storage.

Images do not display

Run php artisan storage:link. In Blade, use Storage::url($product->image_path) for the image URL. Confirm that your public disk is configured and writable.

Watermark missing or image format fails

Confirm public/images/watermark.png exists and the PHP image extension supports the input image and WebP output.

Security note

These resource routes are open for a classroom demo. In a real application, protect editing and destructive actions with authentication and authorization. Permanent deletion is irreversible.

Official documentation

https://image.intervention.io/v4/getting-started/frameworks
https://image.intervention.io/v4/modifying-images/inserting
https://image.intervention.io/v4/basics/image-output
https://laravel.com/docs/eloquent#soft-deleting